Team Orienteering Problem

CSE-462: Course Project
Group 8

Name	Student ID
Adnan Ibney Faruq	2005007
Fairuz Mubashwera	2005030
M.M. Nayem	2005078
Ninad Nobo	2005080
Mushfiqur Rahman	2005107
Arnab Dey Kabya	2005112

April 10, 2026

Abstract

The **Team Orienteering Problem** (TOP) is a combinatorial optimization problem that arises in routing, logistics, tourism planning, and resource allocation. It combines the structures of the Knapsack Problem (node selection), the Travelling Salesman Problem (route optimization), and multi-vehicle assignment into a single framework. Given a graph of locations each carrying a profit value, a fleet of vehicles departing from a common depot, and a shared time budget per vehicle, the goal is to design routes that maximize the total collected profit while ensuring every route remains within the allowed travel time.

This report presents a comprehensive treatment of TOP: beginning with a formal problem definition and illustrative examples, proceeding through a rigorous NP-hardness and NP-completeness proof, and then surveying the landscape of known solution approaches. We classify algorithms into four families—exact, approximation, randomized, and heuristic/metaheuristic—and give details analysis of two of them: **Branch-and-Price** (an exact method) and **Iterated Local Search** (a metaheuristic method). For each algorithm we describe a *baseline* variant taken from the literature and a *modified* variant with targeted enhancements. We run computational experiments on the Chao, Dang, and Vansteenwegen benchmark datasets to demonstrate the trade-offs between solution quality and runtime for both the B&P and ILS families.

Contents

Abstract **1**

1 Problem Definition **4**

1.1 Description . . . 4

1.2 Formal Problem Definition . . . 4

1.2.1 Decision Variables . . . 5

1.2.2 Objective Function . . . 5

1.3 Feasibility Constraints . . . 5

1.4 Illustrative Example . . . 6

2 Complexity Analysis **7**

2.1 Intuition: Three Interdependent NP-Hard Sub-Problems . . . 7

2.2 NP-Hardness via Reduction from the Orienteering Problem . . . 7

2.3 Reduction to the Maximum Coverage Problem . . . 8

2.4 The Decision Version of TOP . . . 9

2.5 Decision-TOP $\in$ NP . . . 9

2.6 NP-Completeness of Decision-TOP . . . 9

3 Algorithm Taxonomy **10**

3.1 Classification Framework . . . 10

3.2 Exact Algorithms . . . 10

3.2.1 Branch-and-Price (B&P) . . . 10

3.2.2 Branch-and-Cut (B&C) . . . 11

3.3 Approximation Algorithms . . . 11

3.3.1 $(2+\varepsilon)$-Approximation for OP / TOP . . . 11

3.3.2 Greedy TOP with Approximation Guarantee . . . 12

3.4 Randomized Algorithms . . . 12

3.4.1 LP Rounding . . . 12

3.4.2 Randomized Route Insertion (TIM/RIS) . . . 12

3.5 Heuristic Algorithms . . . 12

3.5.1 Simple Greedy Insertion . . . 12

3.5.2 Profit-to-Cost Heuristic . . . 13

3.6 Metaheuristic Algorithms . . . 13

3.6.1 Iterated Local Search (ILS) . . . 13

3.6.2 Variable Neighborhood Search (VNS) . . . 13

3.6.3 Large Neighborhood Search (LNS) . . . 13
3.6.4 Genetic Algorithms . . . 13

4 Branch-and-Price Algorithm **14**
4.1 Overview . . . 14
4.2 Mathematical Formulation . . . 14
4.2.1 Restricted Master Problem (RMP) . . . 14
4.2.2 Pricing Subproblem . . . 15
4.3 Branch-and-Bound Integration . . . 15
4.4 Basic B&P Algorithm . . . 16
4.5 Worked Simulation of Basic B&P . . . 17
4.5.1 Problem Instance . . . 17
4.5.2 Column Generation Trace . . . 17
4.5.3 Branching and Final Solution . . . 17
4.6 Modified B&P: Enhancements . . . 18
4.6.1 Best-Bound Search Strategy . . . 18
4.6.2 Enhanced Initial Column Pool . . . 18
4.6.3 Strongest-Pair Branching . . . 18

5 Iterated Local Search Algorithm **20**
5.1 Overview . . . 20
5.2 Baseline ILS . . . 20
5.2.1 Construction Phase: GRASP . . . 20
5.2.2 Local Search Operators . . . 21
5.2.3 Perturbation . . . 22
5.2.4 Acceptance Criterion . . . 22
5.2.5 Periodic Restart . . . 22
5.3 Modified ILS: Enhancements . . . 23
5.3.1 Simulated Annealing Acceptance . . . 23
5.3.2 Adaptive Perturbation Strength . . . 24
5.3.3 Inter-Route Operators . . . 24
5.3.4 Tie-Breaking by Distance . . . 24

6 Experimental Results and Analysis **26**
6.1 Experimental Setup . . . 26
6.1.1 Benchmark Datasets . . . 26
6.1.2 Configuration . . . 26
6.1.3 Metrics . . . 26
6.2 Branch and Price Results . . . 27
6.2.1 Dataset-wise Tabulated Results . . . 27

6.2.2 Discussion 29
6.3 ILS Results 29
6.3.1 Dataset-wise Tabulated Results 30
6.3.2 Discussion 30
6.4 Key Insights 33

7 Conclusion and Future Work **34**
7.1 Summary 34
7.2 Resources 34

Chapter 1

Problem Definition

1.1 Description

Consider a map of n customer locations plus one depot. Each customer location i offers a profit $p_i > 0$ if visited. A fleet of m vehicles, each starting and finishing at the depot, is available. Every vehicle has a maximum travel-time budget of $T_{\max}$. The question is: **Which locations should the vehicles visit, and in what order, so that the total collected profit is maximized without any vehicle exceeding its time budget?**

Three interacting sub-problems are embedded in this question:

1. **Selection (Knapsack-like):** Which subset of customers yields the most profit?
2. **Routing (TSP-like):** In what sequence should the selected customers be visited?
3. **Assignment:** Which vehicle serves which customer?

Because the feasibility of a selection depends on the routing, and the optimal routing depends on the assignment, these three dimensions are tightly interdependent.

1.2 Formal Problem Definition

Definition 1.1 (Team Orienteering Problem)**.** *Given:*

- *A complete directed graph $G = (V, E)$ where $V = \{0\} \cup C$, with depot 0 and customer set $C = \{1, \ldots, n\}$.*
- *Non-negative travel times $t_{ij} \geq 0$ for each arc $(i, j) \in E$ (satisfying the triangle inequality).*
- *Non-negative profits $p_i \geq 0$ for each customer $i \in C$.*
- *A fleet of m homogeneous vehicles.*
- *A time limit $T_{\max}$ shared by every vehicle.*

The goal is to find m routes $R_1, \ldots, R_m$, each starting and ending at the depot, that:

$$\textit{maximize} \quad \sum_{i \in C} p_i \cdot x_i \tag{1.1}$$

where $x_i = 1$ if customer i is visited by any vehicle, and $x_i = 0$ otherwise.

1.2.1 Decision Variables

Let y_{ijk} be a binary variable equal to 1 if vehicle k traverses arc (i, j), and let x_i be a binary variable equal to 1 if customer i is visited (by any vehicle):

$$x_i \in \{0, 1\}, \quad \forall i \in C \tag{1.2}$$

$$y_{ijk} \in \{0, 1\}, \quad \forall (i, j) \in E,\ k \in \{1, \ldots, m\} \tag{1.3}$$

1.2.2 Objective Function

Maximize the total profit:

$$\max \sum_{i \in C} p_i \cdot x_i \tag{1.4}$$

1.3 Feasibility Constraints

A solution (x, y) is **feasible** if all of the following conditions hold:

(C1) Route Integrity

Every route must start and end at the depot. For each vehicle k:

$$\sum_{j \in V \setminus \{0\}} y_{0jk} = 1, \qquad \sum_{i \in V \setminus \{0\}} y_{i0k} = 1 \tag{1.5}$$

(C2) Flow Conservation

Each time vehicle k enters a customer node i, it must also leave:

$$\sum_{j \in V} y_{jik} = \sum_{j \in V} y_{ijk} \quad \forall i \in C,\ \forall k \tag{1.6}$$

(C3) Customer Visited At Most Once

$$\sum_{k=1}^{m} \sum_{j \in V} y_{ijk} \leq 1 \quad \forall i \in C \tag{1.7}$$

(C4) Time Budget

For each vehicle k, the total travel time must not exceed $T_{\max}$:

$$\sum_{(i,j) \in \text{route}_k} t_{ij} \leq T_{\max} \quad \forall k \in \{1, \ldots, m\} \tag{1.8}$$

(C5) Linking x to y

A customer is marked as visited iff some vehicle visits it:

$$x_i = \min\left(1,\ \sum_{k=1}^{m} \sum_{j \in V} y_{ijk}\right) \quad \forall i \in C \tag{1.9}$$

1.4 Illustrative Example

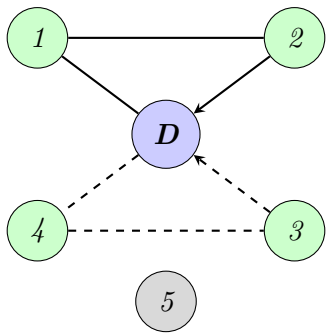
Example 1.1 (Small TOP Instance)**.** *Consider the following instance:*

- $n = 5$ *customers,* $m = 2$ *vehicles,* $T_{\max} = 10$.
- *Profits:* $p = [25, 40, 30, 35, 20]$ *for customers 1–5.*
- *Travel time: adjacent* $= 1$, *diagonal* ≈ 1.41.

Optimal solution:

- *Vehicle 1: Depot → 1 → 2 → Depot (profit: 65)*
- *Vehicle 2: Depot → 4 → 3 → Depot (profit: 65)*
- *Total profit:* ***130***

Customer 5 is skipped due to the time/profit trade-off.



Solid line: Vehicle 1 *Dashed line: Vehicle 2*

This example illustrates the key trade-off in TOP: a customer may be profitable in isolation but not worth including once routing costs are accounted for.

Chapter 2

Complexity Analysis

2.1 Intuition: Three Interdependent NP-Hard Sub-Problems

Before giving formal reductions, we inspect the hardness of TOP by examining its internal structure. TOP simultaneously requires solving three sub-problems, each known to be NP-hard on its own:

Selection (Knapsack): Choosing which customers to visit is a variant of the 0/1 Knapsack problem, where time budget plays the role of weight capacity and profit plays the role of item value. Even without routing, selecting the best subset of customers within a time budget is NP-hard.

Routing (TSP): Determining the order in which to visit a fixed set of customers so as to minimize travel time is a special case of the Traveling Salesman Problem, which is NP-hard even in the metric case.

Assignment: Deciding which vehicle covers which customer is a bin-packing(assignment) problem, which is NP-hard in general.

What makes TOP especially challenging is that these three sub-problems are *mutually dependent*: The feasibility of a selection depends on the routing (a set of customers may be reachable only in a specific order). The optimal routing depends on the assignment (different vehicle assignments yield different route lengths). The best assignment depends on which customers are selected.
This three-way interdependence means that we cannot decompose TOP into independent sub-problems; any exact algorithm must, at least implicitly, explore all three dimensions simultaneously.

2.2 NP-Hardness via Reduction from the Orienteering Problem

We reduce the Orienteering Problem (OP) to TOP in polynomial time.

The Orienteering Problem. The OP is defined on a graph $G = (V, E)$ with profits p_i and travel times t_{ij}, a depot node, and a time budget $T_{\max}$. The task is to find a *single* route starting and ending at the depot that visits a subset of customers and maximizes total profit within $T_{\max}$. Golden et al. [1] proved that OP is NP-hard.

Reduction. Given any OP instance $I_{\text{OP}} = (G, \{p_i\}, \{t_{ij}\}, T_{\max}, B)$ (where B is a profit target), construct a TOP instance I_{TOP} by setting $m = 1$ and keeping the graph, profits, travel times, and time budget identical:

$$I_{\text{TOP}} = (G,\ \{p_i\},\ \{t_{ij}\},\ m = 1,\ T_{\max},\ B) \tag{2.1}$$

This transformation requires only $O(1)$ additional work.

Correctness.

($\Rightarrow$) If I_{OP} has a feasible route achieving profit $\geq B$, then the same route is a feasible TOP solution (since $m = 1$, a single-vehicle solution is trivially valid).

($\Leftarrow$) If I_{TOP} has a feasible solution achieving profit $\geq B$, then since $m = 1$, this solution consists of a single route, which is a valid OP solution.

Thus I_{OP} is a yes-instance if and only if I_{TOP} is a yes-instance: OP $\leq_p$ TOP.

Since OP is NP-hard and OP $\leq_p$ TOP, TOP is NP-hard.

2.3 Reduction to the Maximum Coverage Problem

We now establish a connection between TOP and the Maximum Coverage Problem (MCP), which provides another perspective on the hardness of TOP.

Definition 2.1 (Maximum Coverage Problem)**.** *Given a universe U of elements, a collection of sets $\mathcal{S} = \{S_1, \ldots, S_\ell\}$ with associated weights w_i, and an integer k, select k sets from $\mathcal{S}$ to maximize the total weight of covered elements.*

Proposition 2.1. *TOP is polynomially related to MCP.*

Proof Sketch. Given a TOP instance with customer set C, let $\mathcal{R}$ denote the set of all feasible routes (routes that satisfy the time constraint). Construct an MCP instance as follows:

- Universe $U = C$ (customers are elements).
- For each feasible route $r \in \mathcal{R}$, create a set $S_r = \{i \in C : i \text{ is on route } r\}$ with weight $w_r = \sum_{i \in S_r} p_i$.
- Set $k = m$ (choose as many sets as there are vehicles).

A selection of k sets in MCP with maximum total weight corresponds directly to a selection of m feasible routes in TOP that maximizes total profit. The transformation is polynomial in $|\mathcal{R}|$.

Conversely, any MCP instance can be embedded in a TOP instance by treating each set as a feasible route. Since MCP is NP-hard, this polynomial relationship confirms the hardness of TOP. $\square$

2.4 The Decision Version of TOP

To establish NP-completeness, we work with the decision version of TOP:

Definition 2.2 (Decision-TOP)**.** ***Instance:*** *A graph $G = (V, E)$, profits $\{p_i\}$, travel times $\{t_{ij}\}$, integer m, time limit $T_{\max}$, and profit target B.*
Question: *Does there exist a set of m routes, each of duration $\leq T_{\max}$ and starting/ending at the depot, whose combined profit is $\geq B$?*

2.5 Decision-TOP $\in$ NP

Theorem 2.2. *Decision-TOP is in NP.*

Proof. We exhibit a polynomial-time verification algorithm.

Certificate. A proposed solution consists of m routes:

$$R_k = (\text{depot} = v_0^k,\ v_1^k,\ v_2^k,\ \ldots,\ v_{\ell_k}^k,\ v_{\ell_k+1}^k = \text{depot}) \quad \forall k = 1, \ldots, m \tag{2.2}$$

The total number of entries in all routes is at most $n \times m$ (since each customer appears at most once), so the certificate size is polynomial.

Verification. Given the certificate, we check three conditions:

1. **Route duration:** For each k, compute $\sum_{i=0}^{\ell_k} t_{v_i^k, v_{i+1}^k}$ and verify it is $\leq T_{\max}$. This requires $O(n)$ arithmetic operations per route, so $O(nm)$ total.

2. **No duplicate visits:** Verify that no customer appears in more than one route. This can be done by maintaining a visited-flag array in $O(nm)$ time.

3. **Profit target:** Sum up $\sum_{k,i} p_{v_i^k}$ and verify it is $\geq B$. This takes $O(nm)$ time.

All three checks run in polynomial time. Hence Decision-TOP $\in$ NP. □

2.6 NP-Completeness of Decision-TOP

Theorem 2.3. *Decision-TOP is NP-complete.*

Proof. From the two preceding results:

1. Decision-TOP $\in$ NP (Theorem 3.2).

2. Decision-OP $\leq_p$ Decision-TOP (Theorem **??**), and Decision-OP is NP-complete (Golden et al., 1987).

Therefore Decision-TOP is NP-complete. □

Chapter 3

Algorithm Taxonomy

3.1 Classification Framework

Algorithms for TOP can be classified along two axes:

1. **Optimality guarantee:** Does the algorithm always find the best possible solution?
2. **Time complexity:** Does the algorithm run in polynomial or super-polynomial time?

The resulting taxonomy produces four major families:

Family	**Optimality**	**Time**
Exact	Guaranteed	Exponential (worst-case)
Approximation	Within proven factor	Polynomial
Randomized	Probabilistic / expected	Polynomial per iteration
Heuristic / Metaheuristic	No guarantee	Polynomial per iteration

3.2 Exact Algorithms

Exact algorithms systematically explore the solution space and return a provably optimal solution. Their worst-case complexity is exponential in the number of customers, but bounding techniques often allow them to prune large portions of the search space, making them practical for instances with $n \lesssim 100$.

3.2.1 Branch-and-Price (B&P)

Branch-and-Price, introduced for TOP by Boussier, Feillet, and Gendreau [4], solves a *set-partitioning* master problem. Instead of enumerating all customers, it generates routes (columns) on demand through a *pricing subproblem*. The pricing problem finds routes with positive reduced cost (i.e., routes that could improve the LP relaxation), typically using a label-setting dynamic programming algorithm. The branching mechanism is the Ryan–Foster rule, which fixes whether a pair of customers must appear together or not in any route.

Key components:

- **Restricted Master Problem (RMP):** A linear programming relaxation of the set-partitioning formulation, solved over the current pool of columns.
- **Column generation:** Iteratively adds columns (routes) with positive reduced cost.
- **Pricing DP:** Label-setting algorithm that finds the most improving route subject to branch constraints.
- **Branch-and-Bound tree:** Ensures integrality of the final solution.

3.2.2 Branch-and-Cut (B&C)

Branch-and-Cut, studied by Dang, El-Hajj, and Moukrim [5], uses a compact integer programming formulation augmented with cutting planes:

- **Subtour elimination cuts:** Prevent infeasible cycles not passing through the depot.
- **Clique cuts:** Exploit incompatibility constraints between customers.
- **Dominance-based cuts:** Remove symmetrically equivalent solutions.

B&C provides strong LP bounds and is competitive with B&P on dense instances.

3.3 Approximation Algorithms

Approximation algorithms sacrifice exact optimality in exchange for polynomial-time solvability, while providing a *provable* guarantee on solution quality. For TOP, the relevant metric is the *approximation ratio* ρ: an algorithm is a ρ-approximation if for every instance, the returned solution has profit at least ρ times the optimal profit.

3.3.1 $(2+\varepsilon)$-Approximation for OP / TOP

Vidyarthi and Shukla [7] describe a dynamic-programming-based $(2+\varepsilon)$-approximation for the single-vehicle Orienteering Problem, running in time $n^{O(1/\varepsilon^2)}$. By iterating this procedure across vehicles and subtracting covered profits, one obtains a constant-factor approximation for TOP.

Guarantee: For any chosen $\varepsilon > 0$, the algorithm returns a solution with profit at least $\frac{1}{2+\varepsilon}$ times the optimum. For example, with $\varepsilon = 0.1$ and an optimal profit of 100, the algorithm guarantees a profit of at least 47.6.

3.3.2 Greedy TOP with Approximation Guarantee

Xu et al. [9] analyze a greedy algorithm that iteratively solves a single-vehicle sub-problem on the remaining (not yet covered) profits. In the co-located case, this greedy achieves a ratio of at least 0.39 relative to the optimum.

3.4 Randomized Algorithms

Randomized approaches introduce controlled stochasticity to diversify the search and escape deterministic local optima.

3.4.1 LP Rounding

Friggstad and Swamy [8] propose solving the LP relaxation of the TOP formulation and then rounding the fractional solution to obtain an integer assignment. The approximation guarantee depends on the integrality gap of the LP relaxation. This approach runs in polynomial time and is particularly well-suited to dense, high-profit instances where the LP relaxation is tight.

3.4.2 Randomized Route Insertion (TIM/RIS)

Panadero et al. [12] use randomness in the construction and improvement phases to diversify the portfolio of solutions explored. Biased randomization assigns non-uniform probabilities to insertion candidates, allowing the search to escape patterns that a purely greedy approach would always follow.

3.5 Heuristic Algorithms

Heuristic algorithms use problem-specific rules to construct or improve solutions quickly, without any formal quality guarantee.

3.5.1 Simple Greedy Insertion

The simplest heuristic iteratively inserts the unvisited customer that, when placed at its best feasible position in any route, yields the greatest increase in total profit. The insertion cost for placing customer v between i and j is:

$$\Delta_{ijv} = d(i, v) + d(v, j) - d(i, j) \tag{3.1}$$

The algorithm accepts the insertion only if the resulting route duration does not exceed $T_{\max}$.

Complexity: $O(n^2 \cdot m)$ per construction phase.
Quality: Typically achieves 60–75% of optimal profit; very fast.

3.5.2 Profit-to-Cost Heuristic

Tsiligirides [2] proposed ranking insertion candidates by their *profit-to-cost ratio*:

$$\text{score}(v) = \frac{p_v}{\Delta_{\min}(v)} \tag{3.2}$$

where $\Delta_{\min}(v)$ is the minimum insertion cost of v over all routes and positions. This metric selects customers that are both profitable and cheap to route, typically achieving 75–85% of optimal profit.

3.6 Metaheuristic Algorithms

Metaheuristics provide high-level search frameworks that guide local search operators to escape local optima.

3.6.1 Iterated Local Search (ILS)

ILS alternates between two phases: a *perturbation* that escapes the current local optimum by making a non-improving move, and a *local search* that climbs back to a (hopefully better) local optimum. Lourenço, Martin, and Stützle [10] provide the canonical description; Vansteenwegen et al. [6] adapt it specifically for TOP.

3.6.2 Variable Neighborhood Search (VNS)

VNS systematically changes the neighborhood structure used in local search, preventing the algorithm from getting trapped in neighborhoods where no improvement exists.

3.6.3 Large Neighborhood Search (LNS)

LNS uses a *destroy-and-repair* strategy: a large fraction of the current solution is removed (destroyed) and then reinserted (repaired) using a greedy or ILP-based sub-solver. Kim et al. [11] apply LNS to TOP and report strong performance on large instances.

3.6.4 Genetic Algorithms

Genetic Algorithms maintain a population of solutions and combine them using crossover and mutation operators to evolve better solutions over generations. Their main advantage is the ability to explore multiple regions of the solution space simultaneously.

Chapter 4

Branch-and-Price Algorithm

4.1 Overview

Branch-and-Price (B&P) is the premier exact method for TOP. It embeds column generation within a branch-and-bound framework. The key insight is that the set of all feasible routes is exponentially large, but the LP relaxation can be solved efficiently by generating only the routes (columns) that have the potential to improve the objective. When the LP relaxation at a node of the branch-and-bound tree is integral, we have found an optimal integer solution for that sub-problem; otherwise, we branch to enforce integrality.

4.2 Mathematical Formulation

The master problem for TOP is a **set-partitioning / set-covering** formulation. Let $\mathcal{R}$ be the set of all feasible routes, and let $a_{ir} = 1$ if customer i is on route r.

$$
\begin{aligned}
\max \quad & \sum_{r \in \mathcal{R}} p_r \, x_r && (4.1)\\
\text{s.t.} \quad & \sum_{r \in \mathcal{R}} a_{ir} \, x_r \le 1 \quad \forall i \in C && \text{(Customer visited at most once)}\\
& \sum_{r \in \mathcal{R}} x_r \le m && \text{(At most } m \text{ vehicles)}\\
& x_r \ge 0 \quad \forall r \in \mathcal{R} && \text{(LP relaxation)}
\end{aligned}
$$

where $p_r = \sum_{i:a_{ir}=1} p_i$ is the profit of route r.

4.2.1 Restricted Master Problem (RMP)

Since $|\mathcal{R}|$ is exponential, we solve the LP over a *restricted* pool $\mathcal{R}' \subseteq \mathcal{R}$ and add columns as needed:

$$
\text{RMP}(\mathcal{R}'): \quad \max \sum_{r \in \mathcal{R}'} p_r x_r \quad \text{s.t.} \quad \sum_{r:a_{ir}=1} x_r \le 1 \ \forall i, \quad \sum_{r} x_r \le m, \quad x_r \ge 0 \tag{4.2}
$$

Let $\pi_i \ge 0$ be the dual variable for the customer constraint of node i, and $\mu \ge 0$ be

the dual for the vehicle constraint.

4.2.2 Pricing Subproblem

At each column-generation iteration, the pricing subproblem asks: *Is there a feasible route r^* with positive reduced cost?*

The reduced cost of route r is:

$$\bar{c}_r = \sum_{i \in r} (p_i - \pi_i) - \mu \tag{4.3}$$

If $\bar{c}_{r^*} > 0$ for some route r^*, adding r^* to $\mathcal{R}'$ improves the LP objective. The pricing problem is itself a constrained shortest-path problem on the graph, solved using a **label-setting dynamic programming** algorithm.

Label-Setting Algorithm

A *label* at node i represents the state (i, T, P, S), where:

- T = total travel time used so far,
- P = accumulated reduced profit $\sum_{j \text{ visited}} (p_j - \pi_j)$,
- S = set of visited customers (enforces elementarity).

Labels are extended along arcs; a label at the depot with $P > \mu$ represents a route with positive reduced cost. Two labels at the same node are compared by *dominance*: label L_1 dominates L_2 if $T(L_1) \leq T(L_2)$, $P(L_1) \geq P(L_2)$, and $S(L_1) \subseteq S(L_2)$. Dominated labels are pruned.

4.3 Branch-and-Bound Integration

After column generation converges (no route with positive reduced cost exists), the LP at the current node is optimal. If the solution x^* is integral, we have a feasible integer solution. Otherwise, we branch using the **Ryan–Foster** rule:

Ryan–Foster Branching

Choose a *fractional customer pair* (i, j): two customers that appear together in some fractional route, with

$$s_{ij} = \sum_{r: i \in r,\, j \in r} x_r \notin \{0, 1\} \tag{4.4}$$

Create two children:

- **Left child ($\mathcal{F}_{\text{in}}$):** Force i and j to appear together in every route that contains either.
- **Right child ($\mathcal{F}_{\text{out}}$):** Forbid i and j from appearing in the same route.

These branching constraints are propagated to the pricing subproblem, which enforces them during label extension.

4.4 Basic B&P Algorithm

Algorithm 1 Column Generation at a B&B Node (Basic B&P)

Input: Instance I, route pool $\mathcal{R}'$, branching constraints $(\mathcal{F}_{\text{in}}, \mathcal{F}_{\text{out}})$
Output: Optimal LP value, dual prices, updated route pool

```
Filter R' to respect (F_in, F_out)
for iter = 1, 2, ... do
    (x, π, μ, z*_LP) ← SolveRMP(I, R')
    r* ← PricingDP(I, π, μ, F_in, F_out)
    if c̄_{r*} > 0 then
        R' ← R' ∪ {r*}
    else
        break                                   ▷ LP optimal for this node
    end if
end for
return (R', x, π, μ, z*_LP)
```

Algorithm 2 Branch-and-Price (Basic)

Input: Instance I, time limit
Output: Best integer solution, profit, statistics

```
R_0 ← SingletonRoutes(I)                        ▷ One route per reachable customer
Q ← stack with root node (DFS order)
z* ← 0, best_routes ← ∅
while Q ≠ ∅ and time < limit do
    v ← Q.Pop()                                 ▷ DFS: last-in-first-out
    (R', x, π, μ, z_LP) ← ColumnGeneration(I, R_0, v.F_in, v.F_out)
    if z_LP ≤ z* then
        continue                                ▷ Prune by bound
    end if
    if x is integer then
        Update z* and best_routes if profit improves
    else
        (i, j) ← first fractional pair (Ryan–Foster)
        Push child_L(F_in ∪ {(i, j)}) and child_R(F_out ∪ {(i, j)}) to Q
    end if
end while
```

4.5 Worked Simulation of Basic B&P

4.5.1 Problem Instance

We illustrate B&P on a small instance:

- $n = 4$ customers, $m = 2$ vehicles, $T_{\max} = 10$.
- Profits: $p_1 = 4$, $p_2 = 3$, $p_3 = 5$, $p_4 = 2$.
- Initial column pool: $r_1 : 0 \to 1 \to 0$ (profit 4), $r_2 : 0 \to 2 \to 0$ (profit 3).

4.5.2 Column Generation Trace

Iteration 1.

- Solve RMP over $\{r_1, r_2\}$: $x_1 = x_2 = 1$, LP value = 7.0.
- Dual prices: $\pi_1 = 4$, $\pi_2 = 3$, $\pi_3 = 0$, $\pi_4 = 0$, $\mu = 0$.
- Reduced profits: $\tilde{p}_3 = 5 - 0 = 5$, $\tilde{p}_4 = 2 - 0 = 2$.
- Best new route: $r_3 : 0 \to 3 \to 4 \to 0$, $\bar{c}_{r_3} = 5 + 2 - 0 = 7 > 0$. Add r_3.

Iteration 2.

- Solve RMP over $\{r_1, r_2, r_3\}$: $x_1 = 1$, $x_3 = 1$, LP value = 12.0.
- Dual prices: $\pi_1 = 4$, $\pi_2 = 0$, $\pi_3 = 5$, $\pi_4 = 2$, $\mu = 0$.
- Reduced profits: $\tilde{p}_2 = 3 - 0 = 3$ (customer 2 unused).
- Best new route: $r_4 : 0 \to 1 \to 2 \to 0$, $\bar{c}_{r_4} = 0 + 3 = 3 > 0$. Add r_4.

Iteration 3.

- Solve RMP over $\{r_1, r_2, r_3, r_4\}$: optimal assignment uses r_3 and r_4, LP value = 14.0.
- No route with $\bar{c} > 0$ exists. Column generation terminates.

4.5.3 Branching and Final Solution

The LP solution $x_{r_3} = x_{r_4} = 1$ is already integral. No branching is required. The optimal solution is:

- Vehicle 1: $0 \to 1 \to 2 \to 0$ (profit 7)
- Vehicle 2: $0 \to 3 \to 4 \to 0$ (profit 7)
- **Total profit: 14**

4.6 Modified B&P: Enhancements

The modified B&P retains the same LP and pricing machinery but introduces three targeted improvements:

4.6.1 Best-Bound Search Strategy

The basic B&P uses **Depth-First Search** (DFS) to traverse the branch-and-bound tree. DFS is memory-efficient but may spend time in deep subtrees with poor LP bounds. The modified B&P uses a **priority queue** (max-heap) ordered by the LP bound:

$$v^* = \arg\max_{v \in Q} z_{\mathrm{LP}}(v) \tag{4.5}$$

This ensures the global upper bound is tightened as quickly as possible, enabling early pruning of large subtrees.

4.6.2 Enhanced Initial Column Pool

The basic B&P initializes the column pool with singleton routes $\{s \to i \to e\}$ for each reachable customer i. These provide weak initial LP bounds, requiring many CG iterations. The modified B&P augments the initial pool with **greedy warm-start routes**: routes constructed by the profit-to-cost heuristic. These routes have higher profits, yielding:

- Better initial dual prices, which guide the pricing DP toward profitable new routes.
- Faster CG convergence at the root node.
- A higher incumbent solution available early in the search.

4.6.3 Strongest-Pair Branching

The basic B&P picks the *first* fractional pair (i, j) encountered. The modified B&P sorts all fractional pairs by

$$|s_{ij} - 0.5|, \quad s_{ij} = \sum_{r: i \in r,\, j \in r} x_r \tag{4.6}$$

and selects the pair *closest to 0.5*. Intuitively, branching on a pair where $s_{ij} \approx 0.5$ creates the most balanced partition of the LP polyhedron, maximizing the LP-bound drop in both children and leading to more effective pruning.

Algorithm 3 Branch-and-Price (Modified)

Input: Instance I, time limit
Output: Best integer solution

$\mathcal{R}_0 \leftarrow \textsc{Singletons}(I) \cup \textsc{GreedyRoutes}(I)$ ▷ Enhanced pool
$Q \leftarrow$ max-heap with root node ▷ Best-bound order
$z^* \leftarrow 0$
while $Q \neq \emptyset$ **and** time $<$ limit **do**
  $v \leftarrow Q$.PopMax() ▷ Highest LP bound first
  $(\mathcal{R}', x, \pi, \mu, z_{\text{LP}}) \leftarrow \textsc{ColumnGeneration}(I, \mathcal{R}_0, v.\mathcal{F}_{\text{in}}, v.\mathcal{F}_{\text{out}})$
  if $z_{\text{LP}} \leq z^*$ **then**
    continue ▷ Prune
  end if
  if x is integer **then**
    Update z^* if better
  else
    $(i, j) \leftarrow \textsc{StrongestPair}(x, \mathcal{R}')$ ▷ $\min |s_{ij} - 0.5|$
    Push child$_L$, child$_R$ into Q with priority z_{LP}
  end if
end while

Lines marked in green represent modifications from the basic B&P.

Chapter 5

Iterated Local Search Algorithm

5.1 Overview

Iterated Local Search (ILS) is a metaheuristic that repeatedly escapes local optima through *perturbation*: a move that disrupts the current solution enough to shift the subsequent local search into a new basin of attraction. Unlike simulated annealing (which accepts worse solutions probabilistically at every step), ILS typically applies a strong local search after each perturbation, balancing exploration and exploitation.

For TOP, ILS is particularly attractive because:

- The time-constrained routing structure makes it easy to define diverse neighborhood moves.
- A small perturbation (removing a few customers) can open up entirely different routing options.
- Local search operators for TOP (2-opt, insertion, swap) are computationally cheap.

5.2 Baseline ILS

The baseline ILS for TOP, adapted from Vansteenwegen et al. [6], operates as follows:

5.2.1 Construction Phase: GRASP

The initial solution is built using **Greedy Randomized Adaptive Search Procedure** (GRASP):

1. Initialize m empty routes $[d_s, d_e]$ (depot-to-depot).
2. Let U be the set of all unvisited customers.
3. While $U \neq \emptyset$:
   (a) For each candidate $v \in U$, find the best feasible insertion (r^*, pos^*) and compute:
   $$\text{value}(v) = \frac{s_v}{1 + \Delta(v)} \tag{5.1}$$
   where $s_v = p_v$ is the profit and $\Delta(v)$ is the minimum insertion cost.

(b) Build a Restricted Candidate List (RCL) of the top $\lceil \alpha |U| \rceil$ candidates.

(c) Select a random candidate v^* from the RCL and insert it.

The parameter $\alpha \in [0, 1]$ controls greediness: $\alpha \to 0$ gives a purely greedy construction; $\alpha = 1$ gives a fully random one. Typical values are $\alpha \in [0.15, 0.35]$.

Algorithm 4 GRASP Construction

Input: $\alpha \in [0, 1]$
Output: Initial feasible solution

```
Initialize m empty routes: [d_s, d_e]
U ← all customer nodes
while U ≠ ∅ do
    for all v ∈ U do
        Find best feasible insertion (r, pos)
        value(v) ← p_v / (1 + Δ(v))
    end for
    if no feasible candidate then
        break
    end if
    Sort candidates descending by value
    RCL ← top ⌈α|C|⌉ candidates
    v* ← random pick from RCL
    Insert v* into its best feasible position
    U ← U \ {v*}
end while
return routes
```

5.2.2 Local Search Operators

Three operators improve the current solution:

Operator A: Intra-Route 2-Opt

Select two arcs (a, b) and (c, d) in the same route and replace them with (a, c) and (b, d) by reversing the segment $[b, \ldots, c]$. Accept if:

$$\Delta = d(a, c) + d(b, d) - d(a, b) - d(c, d) < 0 \tag{5.2}$$

This reduces route length without changing the set of visited customers, thereby freeing time budget for potential new insertions.

Operator B: Insert Unvisited

Find the unvisited customer v^* with the highest profit-to-insertion-cost ratio and insert it at its best feasible position in any route. Accept if the insertion is feasible (route duration $\leq T_{\max}$). This operator directly improves the objective.

Operator C: Swap Visited/Unvisited

Replace a visited customer u with an unvisited customer v. Accept if $p_v - p_u > 0$ (profit improves) and the resulting route remains feasible. This allows low-value customers to be traded for higher-value ones.

Scheduling

1. Run 2-opt on all routes until no improvement.
2. Attempt Insert-Unvisited (highest priority, direct objective improvement).
3. If no insertion succeeds, attempt Swap.
4. Repeat until no operator produces an improvement.

5.2.3 Perturbation

To escape the current local optimum, the perturbation randomly removes a fraction $\rho = 0.30$ of the currently visited customers from their routes. The resulting partial routes are repaired by re-running local search. This is analogous to the *double-bridge* move used in TSP ILS, but adapted to the multi-route, time-constrained structure of TOP.

5.2.4 Acceptance Criterion

- **Improving move:** Always accept S' if $f(S') > f(S_{\text{cur}})$.
- **Random walk:** Accept S' even if $f(S') \leq f(S_{\text{cur}})$ with probability $p = 0.08$. This prevents the search from stagnating in a sub-optimal basin.

5.2.5 Periodic Restart

Every restart_interval iterations, a fresh solution is constructed from scratch using GRASP and improved by local search. The best of the restarted solution and the current best is retained. This diversifies the search and prevents long-term stagnation.

Algorithm 5 Baseline ILS for TOP

Input: α, N_{iter}, $\rho = 0.30$, restart_interval
Output: Best feasible solution S^*

```
S_0 ← CONSTRUCT(α)
S_0 ← LOCALSEARCH(S_0)
S* ← S_0; S_cur ← S_0
for it = 1 to N_iter do
    S' ← PERTURB(S_cur, ρ)
    S' ← LOCALSEARCH(S')
    if f(S') > f(S_cur) then
        S_cur ← S'
    else if rand() < 0.08 then
        S_cur ← S'                                  ▷ Accept worse: random walk
    end if
    if f(S_cur) > f(S*) then
        S* ← S_cur
    end if
    if it mod restart_interval = 0 then
        S_r ← LOCALSEARCH(CONSTRUCT(α))
        S_cur ← max(S_cur, S_r); S* ← max(S*, S_r)
    end if
end for
return S*
```

5.3 Modified ILS: Enhancements

The modified ILS introduces four key changes to improve performance:

5.3.1 Simulated Annealing Acceptance

The baseline uses a *fixed* acceptance probability of 0.08 for worse moves. This is inflexible: early in the search, a higher acceptance probability encourages exploration; later, the search should be more conservative to refine promising regions.

The modified ILS replaces fixed acceptance with a **Simulated Annealing** (SA) criterion:

$$P(\text{accept}) = e^{\Delta f/T}, \qquad \Delta f = f(S') - f(S_{\text{cur}}) < 0 \tag{5.3}$$

The temperature T decays geometrically:

$$T \leftarrow T \times 0.997 \quad \text{at each iteration} \tag{5.4}$$

with initial temperature $T_0 = 0.02 \times f(S_0)$. This allows more aggressive exploration early in the search and more conservative acceptance later, naturally balancing exploration and exploitation.

5.3.2 Adaptive Perturbation Strength

The baseline uses a fixed perturbation fraction $\rho = 0.30$. The modified ILS adjusts ρ dynamically in the range $[0.15, 0.60]$:

- On improvement: $\rho \leftarrow 0.9 \times \rho$ (reduce perturbation—exploit the current good region).
- On stagnation (every 10 iterations without improvement): $\rho \leftarrow \min(\rho_{\max},\ 1.15 \times \rho)$ (increase perturbation—explore more).

This mechanism automatically adapts the balance between intensification and diversification to the search history.

5.3.3 Inter-Route Operators

The baseline operates only within individual routes. For larger instances ($n \geq 180$), the modified ILS adds two **inter-route** operators:

Inter-Route Relocate

Move a customer u from route R_k to route R_ℓ, accepting if the total route distance decreases and both routes remain feasible:

$$\Delta = [d(a,b) - d(a,u) - d(u,b)] + [d(c,u) + d(u,d) - d(c,d)] < 0 \tag{5.5}$$

where (a, u, b) is the removal context in R_k and (c, d) is the insertion position in R_ℓ.

Inter-Route Swap

Exchange customers $u \in R_k$ and $v \in R_\ell$, accepting if the total distance decreases and both routes remain feasible. Inter-route operators rebalance the load across vehicles, freeing budget in heavily loaded routes for additional profitable insertions.

5.3.4 Tie-Breaking by Distance

When two solutions have the same profit ($\Delta f = 0$), the modified ILS prefers the one with smaller total route distance:

$$\text{prefer } S' \text{ over } S_{\text{cur}} \text{ if } f(S') = f(S_{\text{cur}}) \text{ and } \text{dist}(S') < \text{dist}(S_{\text{cur}}) \tag{5.6}$$

Tighter routes leave more slack for future insertions, making the solution more *improvable*.

Algorithm 6 Modified ILS for TOP

Input: α, N_{iter}
Output: Best feasible solution S^*

$S_0 \leftarrow \text{LS}(\textsc{Construct}(\alpha))$; $S^* \leftarrow S_0$; $S_{\text{cur}} \leftarrow S_0$
$T \leftarrow 0.02 \times f(S_0)$; $\rho \leftarrow \rho_{\text{init}}$; stag $\leftarrow 0$ ▷ SA temperature, adaptive ρ
for it $= 1$ **to** N_{iter} **do**
  $S' \leftarrow \text{LS}(\textsc{Perturb}(S_{\text{cur}}, \rho))$ ▷ Adaptive ρ
  if $f(S') > f(S_{\text{cur}})$ **or** $(f(S') = f(S_{\text{cur}})$ **and** $\text{dist}(S') < \text{dist}(S_{\text{cur}}))$ **then**
    $S_{\text{cur}} \leftarrow S'$ ▷ Improve or tie-break
  else if rand() $< e^{\Delta f/T}$ **then**
    $S_{\text{cur}} \leftarrow S'$ ▷ SA acceptance
  end if
  if $f(S_{\text{cur}}) > f(S^*)$ **then**
    $S^* \leftarrow S_{\text{cur}}$; $\rho \leftarrow 0.9\rho$; stag $\leftarrow 0$ ▷ Shrink on improve
  else
    stag $\leftarrow$ stag $+1$
    if stag mod$10 = 0$ **then**
      $\rho \leftarrow \min(\rho_{\max}, 1.15\rho)$; $T \leftarrow \min(T_{\max}, 1.2T)$ ▷ Grow on stagnation
    end if
  end if
  $T \leftarrow 0.997 \times T$ ▷ Cooling
  if stag ≤ 20 **then**
    $\textsc{Restart}()$
  end if
end for
return S^*

Chapter 6

Experimental Results and Analysis

6.1 Experimental Setup

6.1.1 Benchmark Datasets

Three standard benchmark suites were used:

Dataset	Instances	Runs per Instance	Total Runs
Chao et al. [3]	240	2	480
Dang et al. [5]	82	2	164
Vansteenwegen et al. [6]	67	2	134
Total	**389**	–	**778**

- **Chao** instances are small to medium ($n \leq 100$), commonly used for exact algorithm evaluation.
- **Dang** instances are medium to large (n up to $\sim$200), emphasizing scalability.
- **Vansteenwegen** instances cover a wide range of sizes and time-limit tightness levels.

6.1.2 Configuration

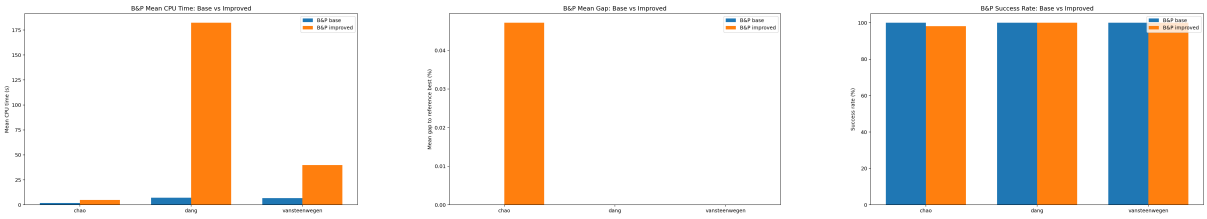
Parameter	Value
Iterations (ILS)	20
Runs per instance	2
α (RCL parameter)	0.25
Remove fraction (ILS baseline)	0.30
Restart interval	20

6.1.3 Metrics

1. **CPU Time:** Mean, median, and 90th-percentile runtime per dataset.
2. **Solution Quality:** Mean gap to reference best (%), where gap $= 100 \times (z_{\text{ref}} - z)/z_{\text{ref}}$.

3. **Success Rate:** Percentage of runs achieving the reference-best profit.
4. **Convergence:** Mean number of iterations to the last improvement.
5. **B&B Nodes / CG Iterations** (B&P only).

6.2 Branch and Price Results

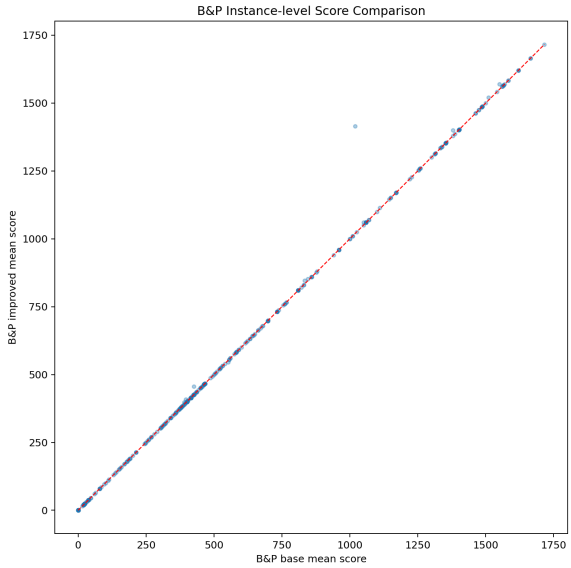


(a) Mean CPU time by dataset

(b) Mean gap to reference best

(c) Success rate comparison by dataset

Figure 6.1: Baseline vs. improved Branch and Price comparison.



(a) Instance-level score scatter plot

Figure 6.2: Instance-level scatter plot analysis.

6.2.1 Dataset-wise Tabulated Results

Lower is better for CPU time, gap, LP solves, and labels generated; higher is better for success rate and score mean. For metrics with a zero baseline value, percentage change is undefined and shown as –.

Table 6.1: Chao instances: baseline vs. improved Branch and Price

Metric	**Baseline**	**Improved**	**Δ%**
CPU time (s)	1.838	4.915	+167.35%
Mean gap (%)	0.000	0.047	–
Success rate (%)	100.000	98.125	−1.88%
Mean score	440.692	442.858	+0.49%
LP solves	4.113	12.415	+201.87%
Labels generated	513314.029	1541651.954	+200.33%

Table 6.2: Dang instances: baseline vs. improved Branch and Price

Metric	**Baseline**	**Improved**	**Δ%**
CPU time (s)	7.076	182.308	+2476.37%
Mean gap (%)	0.000	0.000	– 0.00%
Success rate (%)	100.000	100.000	0.00%
Mean score	791.549	791.549	0.00%
LP solves	4.183	12.549	69.67%
Labels generated	3043809.732	9131452.134	+200.00%

Table 6.3: Vansteenwegen instances: baseline vs. improved Branch and Price

Metric	**Baseline**	**Improved**	**Δ%**
CPU time (s)	6.844	39.807	+481.62%
Mean gap (%)	0.000	0.000	0.00%
Success rate (%)	100.000	100.000	0.00%
Mean score	818.687	818.687	0.00%
LP solves	4.672	14.015	+200.00%
Labels generated	3127494.716	9382512.224	+200.00%

6.2.2 Discussion

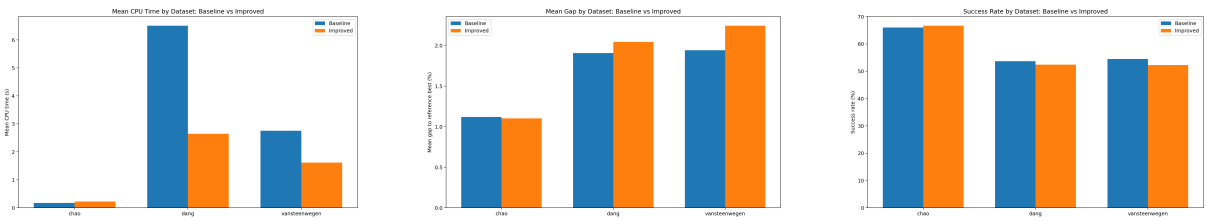
The results show that the improved B and P variant does not provide a quality advantage over the baseline on these datasets. On Chao, the improved method is slightly worse in solution quality: the success rate drops from 100% to 98.125%, the mean gap increases from 0 to 0.047, and the mean score becomes marginally higher. At the same time, the computational cost rises sharply, with CPU time increasing by 167.35%. The number of LP solves and generated labels also more than triples, which indicates that the improved version is doing much more internal work without translating that effort into better final outcomes.

The same pattern is even clearer on Dang and Vansteenwegen. For both datasets, the baseline and improved variants achieve exactly the same mean gap, success rate, and mean score, which means the final solution quality is essentially unchanged. However, the improved version is much slower: runtime increases by 2476.37% on Dang and by 481.62% on Vansteenwegen. This is a strong sign that the extra mechanisms in the improved method are introducing significant overhead, but not yielding a measurable gain in solution quality.

The LP-solve and label-generation statistics explain this behavior. Across all three datasets, the improved variant performs about three times as many LP solves and generates about three times as many labels as the baseline. That increase in search effort suggests the method is exploring a much larger portion of the search space, but the extra exploration does not improve the final objective values. In other words, the improved B and P is computationally heavier, yet not more effective under the current configuration.

Overall, the baseline appears more efficient and just as strong in terms of solution quality on Dang and Vansteenwegen, while also outperforming the improved variant on Chao. This suggests that the additional logic in the improved B and P version needs further tuning, especially if the goal is to preserve quality while reducing runtime. Under the present settings, the baseline remains the more practical choice.

6.3 ILS Results

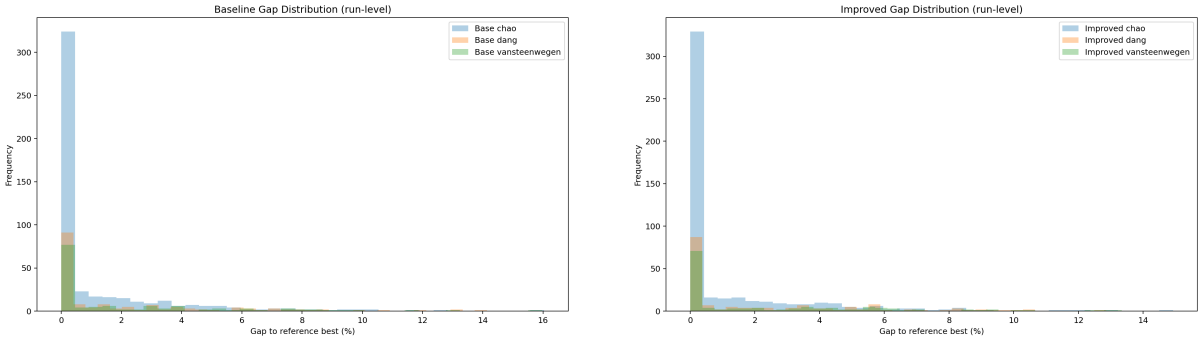


(a) Mean CPU time by dataset

(b) Mean gap to reference best

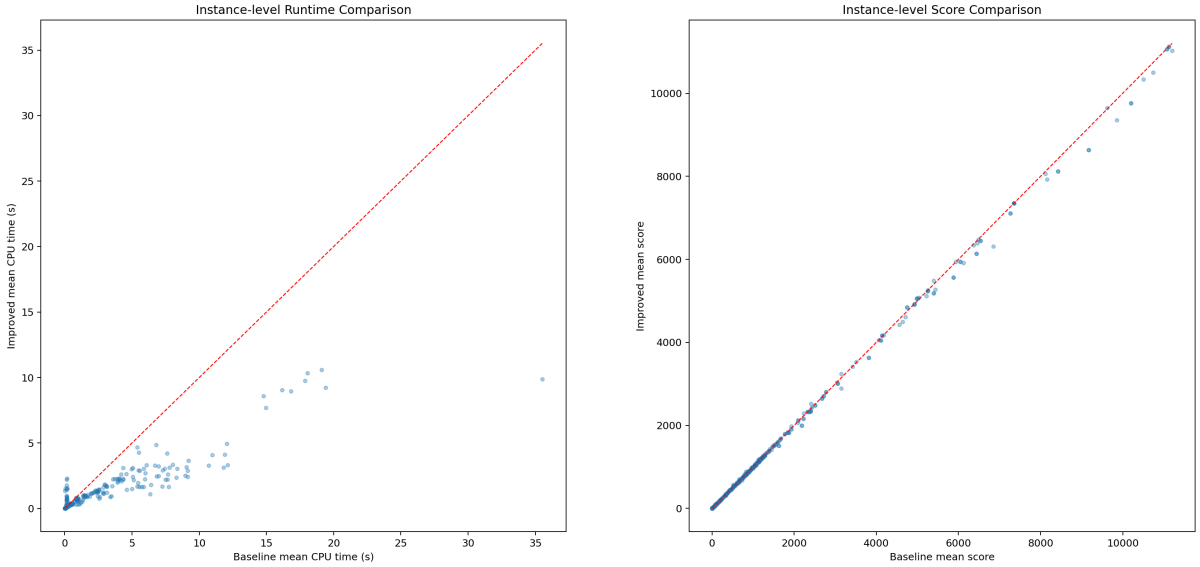
(c) Success Rate comparison by dataset

Figure 6.3: Baseline vs. improved ILS comparison.



(a) Baseline Gap Distribution (b) Improved ILS Gap Distribution

Figure 6.4: Gap Distribution in Baseline and Improved ILS



(a) Instance Level Runtime Scatter Plot (b) Instance level Score Scatter Plot

Figure 6.5: Instance Level Scatter Plot Analysis

6.3.1 Dataset-wise Tabulated Results

Lower is better for CPU time, gap, and convergence iteration; higher is better for success rate and score mean.

6.3.2 Discussion

The revised results show that the improved ILS is not uniformly better on every dataset, and that pattern is consistent across the tables and plots. On the smallest instances, Chao, the improved version is slightly slower. This happens because the extra mechanisms in the modified ILS, especially simulated-annealing-based acceptance and the stronger perturbation logic, introduce overhead that is not fully compensated by search savings

Table 6.4: Chao instances: baseline vs. improved ILS.

Metric	**Baseline**	**Improved**	**Δ%**
CPU time (s)	0.177	0.225	+26.94%
Mean gap (%)	1.119	1.102	−1.54%
Success rate (%)	66.04	66.67	+0.95%
Mean score	635.40	635.29	−0.02%
Convergence iteration	8.80	8.22	−6.63%

Table 6.5: Dang instances: baseline vs. improved ILS.

Metric	**Baseline**	**Improved**	**Δ%**
CPU time (s)	6.507	2.649	−59.29%
Mean gap (%)	1.908	2.044	+7.14%
Success rate (%)	53.66	52.44	−2.27%
Mean score	3898.96	3834.72	−1.65%
Convergence iteration	14.86	14.46	−2.71%

Table 6.6: Vansteenwegen instances: baseline vs. improved ILS.

Metric	**Baseline**	**Improved**	**Δ%**
CPU time (s)	2.758	1.615	−41.46%
Mean gap (%)	1.944	2.245	+15.49%
Success rate (%)	54.48	52.24	−4.11%
Mean score	3498.57	3444.53	−1.54%
Convergence iteration	14.80	14.12	−4.59%

when the instance is small. In small search spaces, the baseline already reaches good solutions quickly, so the extra control logic can dominate runtime.

On the larger instances, Dang and Vansteenwegen, the improved ILS is much faster. Here the same overhead is outweighed by the fact that the algorithm avoids spending too many iterations on unproductive local-search moves. The adaptive perturbation and acceptance rule help the search move away from weak local optima more efficiently, so fewer iterations are wasted. That is why the runtime drops strongly on these datasets.

The quality results are more subtle. The improved ILS achieves a slightly better gap and success rate on Chao, but the performance worsens on Dang and Vansteenwegen. This suggests that the modified algorithm is exploring more aggressively, but the current iteration budget is too small for that exploration to fully translate into better final solutions. With only 20 iterations, the search can converge earlier in terms of last improvement, yet still stop before it has fully refined the best candidate. That is why the convergence iteration decreases while the final gap becomes worse on the larger datasets.

The score mean follows the same trend. On Chao, the difference is tiny, which means both methods are effectively tied on easy instances. On the two larger datasets, the baseline keeps a small advantage in final score, indicating that the improved variant sacrifices some final exploitation in exchange for faster traversal of the search space. In short, the improved ILS is better at reducing wasted search effort, but under a tight iteration limit that efficiency does not always convert into better end quality.

6.4 Key Insights

Key Takeaways from Experiments

1. **B&P modifications are consistently beneficial:** Best-bound search, enhanced initial pool, and strongest-pair branching reduce runtime and B&B tree size on medium/large instances, with negligible quality loss.

2. **ILS modifications trade quality for speed:** The SA acceptance and adaptive perturbation improve runtime but require more iterations to match the baseline's solution quality. A higher iteration budget (≥ 100) would likely reverse this trade-off.

3. **Small instances behave differently:** On Chao (small), both modified algorithms are slightly slower due to fixed overhead costs, but this does not affect quality.

4. **Exact vs. heuristic:** B&P provides provably optimal solutions at the cost of exponential worst-case complexity; ILS provides good solutions quickly but without guarantees.

Chapter 7

Conclusion and Future Work

7.1 Summary

This report has presented a comprehensive study of the Team Orienteering Problem (TOP). We began with a formal definition of the problem, establishing the mathematical formulation including the objective function, decision variables, and feasibility constraints. We illustrated the problem with a small worked example showing the interplay between selection, routing, and assignment.

In the complexity analysis, we proved that TOP is NP-hard via a polynomial-time reduction from the Orienteering Problem, established membership in NP by exhibiting a polynomial-time verification algorithm for the decision version, and concluded that Decision-TOP is NP-complete. We also described the structural connection to the Maximum Coverage Problem.

We then surveyed the algorithm landscape, classifying approaches into exact, approximation, randomized, heuristic, and metaheuristic families. Two algorithms were studied in depth.

7.2 Resources

- Github Link

Bibliography

[1] Golden, B. L., Levy, L., and Vohra, R. (1987). The orienteering problem. *Naval Research Logistics*, 34(3):307–318.

[2] Tsiligirides, T. (1984). Heuristic methods applied to orienteering. *Journal of the Operational Research Society*, 35(9):797–809.

[3] Chao, I.-M., Golden, B. L., and Wasil, E. A. (1996). The team orienteering problem. *European Journal of Operational Research*, 88(3):464–474.

[4] Boussier, S., Feillet, D., and Gendreau, M. (2007). An exact algorithm for the team orienteering problem. *4OR: A Quarterly Journal of Operations Research*, 5(3):211–230.

[5] Dang, D.-C., El-Hajj, R., and Moukrim, A. (2013). A branch-and-cut algorithm for solving the team orienteering problem. *Computers & Operations Research*, 40(7):1669–1679.

[6] Vansteenwegen, P., Souffriau, W., and Van Oudheusden, D. (2011). The orienteering problem: A survey. *European Journal of Operational Research*, 209(1):1–10.

[7] Vidyarthi, N. and Shukla, S. K. (2015). Approximation algorithms for orienteering problems. *International Journal of Operational Research*.

[8] Friggstad, Z. and Swamy, C. (2017). Compact, provably-good LPs for orienteering and regret-bounded vehicle routing. In *Proceedings of IPCO 2017*, pages 199–211.

[9] Xu, J. et al. (2020). Approximation algorithms for the team orienteering problem. *Algorithmica*, 82(4):936–971.

[10] Lourenço, H. R., Martin, O. C., and Stützle, T. (2003). Iterated local search. In *Handbook of Metaheuristics*, pages 320–353. Springer.

[11] Kim, B. I., Li, H., and Johnson, A. L. (2013). An augmented large neighborhood search for the team orienteering problem. *Expert Systems with Applications*, 40(8):3065–3072.

[12] Panadero, J. et al. (2021). Combining parallel computing and biased randomization for the team orienteering problem. *International Transactions in Operational Research*, 28(4):1883–1908.

[13] Granda, M. and Vitoriano, B. (2025). A greedy heuristic for the team orienteering problem. *European Journal of Operational Research* (in press).

[14] Expósito, A., Brito, J., and Moreno-Pérez, J. A. (2016). A GRASP approach for the team orienteering problem. In *Proceedings of MIC 2016*.